```
def index(request):
      html = "Hello world, %s page requested"
  if request.GET.has_key('page_no')
      html = "Hello World, Page requested is no: %s" %request.GET['page_no']
      return HttpResponse(html)
```

Now the page responds according to the GET request variable, page_no. Try different URLs—for example, *http://127.0.0.1:8000/helloweb/?page_id=100* and *http://127.0.0.1:8000/helloweb/*

We will now look at how to handle *page/15/*-type URL mapping to views—for example, *http://127.0.0.1:8000/helloweb/page/44/*. Peeking inside the *urls.py* file, we have:

```
urlpatterns = patterns('',
      (r'^helloweb/page/(?P<page_id>\d+)/$','helloweb.views.page'),
      )
```

The view function in *views.py*:

```
def page(request, page_id):
      return HttpResponse("This is Page: %s" %page_id)
```

Here, *(?P<page_id>\d+)* is the regular expression technique used: *\d+* defines the type of data, i.e., the digits; and *<page_id>* defines the name of the variable, so that we can use it as an argument for the view function page.

The view function can parse the page number from the URL and set *page_id=number*, so that we can manipulate the output *HttpResponse* according to that page number.

Here we have discussed a simple HttpResponse, which always returns HTML code given as an argument. Now we will look at how to deal with templates.

From now, we will place the HTML code as separate *.html* files, instead of inside the view function—and we call them as templates. Set the templates directory in *settings.py*:

```
TEMPLATE_DIRS = (
"/home/slynux/LFY/helloweb/html_templates"
)
```

Create an HTML file in the *html_templates/* directory called *template1.html*, with the following content:

```
<html>
<head><title> Django Hello Web project</title></head>
<body>
<p> This is a helloweb template</p>
<table>
<tr><td>Student </td><td>Roll No</td></tr>
<tr><td>S1</td><td>1</td></tr>
<tr><td>S2 </td><td>2</td></tr>
<tr><td>S3 </td><td>3</td></tr>
</table>
</body>
</html>
```

Now we can modify the view function to support the templates. The HTML interface code is taken through a *get_template* function. Here, *Context()* has a dictionary argument. It takes the variables and objects to be passed to the template to generate dynamic code. However, no objects are passed to the template—the template is just a static HTML page:

```
def index(request):
      from django.template.loader import get_template
      from django.template import Context
      site_template  = get_template('template1.html')
      html = site_template.render(Context({}))
      return HttpResponse(html)
```

Or we can write the same template rendering in one line using the *render_to_response* shortcut as follows:

```
def index(request):
      from django.shortcuts import render_to_response
      return render_to_response('template1.html',{})
```

We will receive an output like what's shown below:

This is a helloweb template

Student	Roll No
S1	1
S2	2
S3	3

This is purely static. Now we will write a dynamic page utilising the facilities of the Django template system. Change the table section of *template1.html* to the following:

```
<table>
<tr><td>Student </td><td>Roll No</td></tr>
{% for name,rollno in student_list.items %}
<tr><td>{{name}}</td><td>{{rollno}}</td></tr>
{% endfor %}
</table>
also add the following code just above the <table>
<p>{% if flag %}
      Flag is on.
{% else %}
      Flag is off.
{% endif %}
</p>
```

...and the new view function to:

```
def index(request):
   from django.shortcuts import render_to_response
   flag=False
   if request.GET.has_key('flag'):
      flag=True
   student_list = {'S1':1, 'S2':2, 'S3':3,'S4':4}
   return render_to_response('template1.html',{'student_list':student_list, 'flag':flag})
```